

APOSTILA DO ALUNO — AULA 5

UML — Diagrama de Classes I

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 24/09/2026 (quinta-feira) | Horário: 07:30 às 11:30 (4h)

Objetivos desta aula

- Rever, de forma breve, o Diagrama de Casos de Uso da Aula 4 (atores, casos de uso, include, extend e generalização).
- Entender a diferença entre casos de uso (o que o sistema faz) e classes (do que o sistema é feito).
- Reconhecer os elementos do Diagrama de Classes: classe, objeto, atributos, métodos e visibilidade (+, -, #).
- Identificar classes candidatas a partir de um texto, usando a técnica dos substantivos.
- Relacionar a visibilidade da UML (+, -, #) com public/private/protected no código.
- Reconhecer os relacionamentos básicos: associação, agregação, composição e herança.
- Desenhar as classes do sistema TechBurger com atributos, métodos e visibilidade.
- Analisar criticamente um diagrama de classes: encontrar, justificar e corrigir erros.

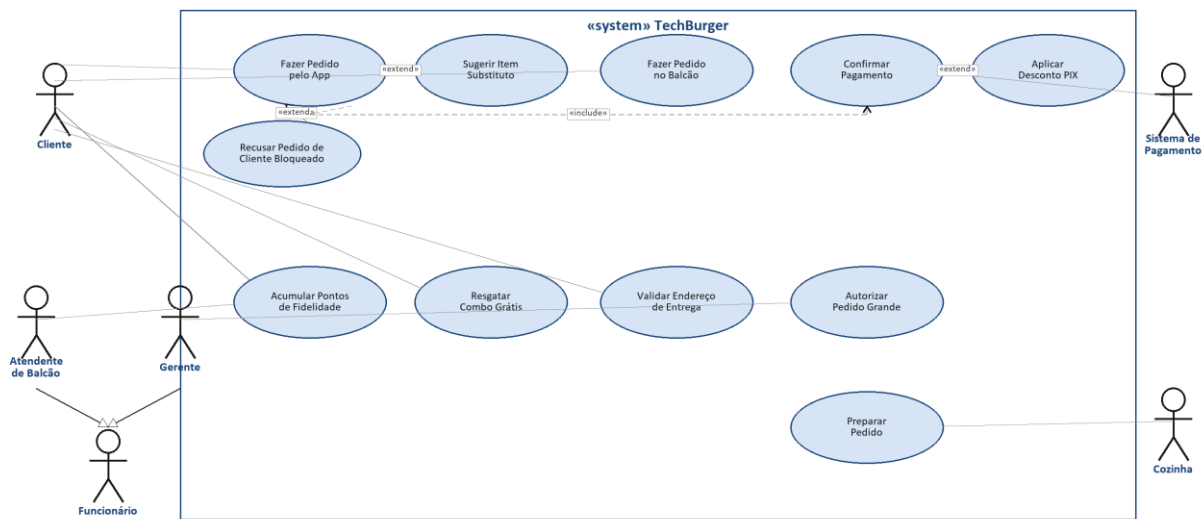
Revisão breve: Aula 4 — Diagrama de Casos de Uso

Antes de ir para as classes, um raio-X rápido do que vimos na aula passada. O Diagrama de Casos de Uso é um diagrama comportamental: mostra O QUE o sistema faz, do ponto de vista de quem o usa. Ele tem dois personagens principais — os atores (quem interage) e os casos de uso (o que o sistema oferece) — e não mostra como o sistema funciona por dentro nem quais dados ele guarda.

Elemento	O que representa	Notação
Ator	Quem ou o que interage com o sistema (pessoa ou outro sistema)	Boneco palito, sempre FORA da fronteira
Caso de uso	Uma funcionalidade que o sistema oferece a um ator	Elipse com verbo no infinitivo + complemento
Sistema (fronteira)	Delimita o que está dentro e o que está fora do sistema	Retângulo envolvendo os casos de uso

- Associação — linha simples entre ator e caso de uso.
- «include» — inclusão obrigatória: o caso incluído sempre acontece dentro do fluxo principal (ex.: “Fazer Pedido pelo App” inclui “Confirmar Pagamento”).
- «extend» — extensão opcional ou condicional (ex.: “Aplicar Desconto PIX” estende “Confirmar Pagamento”).
- Generalização — um ator é um tipo mais específico de outro (ex.: Atendente de Balcão e Gerente são tipos de Funcionário; na Biblioteca “Ler Mais”, Professor é um tipo de Associado).

Regra prática para decidir entre os dois: pergunte “isso acontece sempre (include) ou só às vezes, sob uma condição (extend)?”.



Resposta-modelo da Aula 4: Diagrama de Casos de Uso do TechBurger (5 atores e 10 casos de uso).

Quiz relâmpago — revisão da Aula 4

Testem o que lembram. Respostas no Gabarito.

- 1. O ator fica dentro ou fora da fronteira do sistema?
- 2. “Clicar no botão confirmar” é um bom nome de caso de uso? Por quê?
- 3. “Aplicar Desconto PIX” é include ou extend de “Confirmar Pagamento”? Por quê?
- 4. Se um passo SEMPRE acontece dentro de um fluxo, isso é include ou extend?
- 5. Dê um exemplo de generalização entre atores no TechBurger e um na Biblioteca “Ler Mais”.
- 6. O Diagrama de Casos de Uso mostra quais dados o sistema guarda (cliente, pedido...)?

A última pergunta é a ponte para o assunto de hoje: os casos de uso dizem o que o sistema FAZ; os dados e a estrutura interna ficam para o Diagrama de Classes.

1. Dos Casos de Uso às Classes

	Casos de Uso (Aula 4)	Classes (hoje)
Tipo	Comportamental	Estrutural
Pergunta	O QUE o sistema faz?	DO QUE o sistema é feito?
Ponto de vista	De quem usa o sistema	De quem constrói o sistema
Elementos	Atores, casos de uso, fronteira	Classes, atributos, métodos, relacionamentos
Exemplo TechBurger	Fazer Pedido pelo App	Cliente, Pedido, ItemPedido

Cada caso de uso mobiliza várias classes. Os casos de uso da Aula 4 já sugerem quais classes precisam existir:

Caso de uso (Aula 4)	Classes envolvidas
Fazer Pedido pelo App	Cliente, Pedido, ItemPedido, Produto
Confirmar Pagamento	Pedido, Pagamento
Validar Endereço de Entrega	Pedido, Entrega
Acumular Pontos de Fidelidade	Cliente, Pedido
Autorizar Pedido Grande	Pedido, Gerente (Funcionário)

O que é o Diagrama de Classes

O Diagrama de Classes é um diagrama estrutural: mostra as classes do sistema, seus atributos, seus métodos e como elas se relacionam. É um dos diagramas UML mais usados no mercado e serve de base para o código orientado a objetos e, indiretamente, para o banco de dados. Hoje vemos classe, objeto, atributos, métodos, visibilidade, identificação de classes candidatas e os relacionamentos básicos; a cardinalidade (quantos de cada lado) fica para a Aula 6.

Da regra de negócio ao atributo

As regras de negócio da Aula 1 já contam quais dados o sistema precisa guardar:

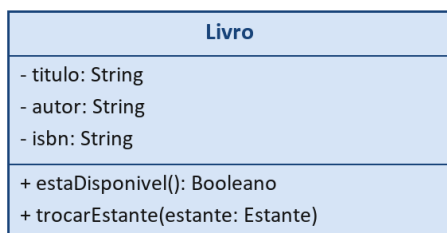
- Regra 1 (valor mínimo de R\$ 15,00 no aplicativo) → o Pedido precisa de valorTotal e tipoEntrega.
- Regra 2 (programa de fidelidade) → o Cliente precisa de pontosFidelidade.
- Regra 5 (entrega só até 5 km) → a Entrega precisa de distanciaKm e do método validarRaio().
- Regra 6 (item em falta no estoque) → o Produto precisa de estoque e dos métodos verificarDisponibilidade() e sugerirSubstituto().
- Regra 8 (3 cancelamentos no mês bloqueiam o app) → o Cliente precisa contar os cancelamentos do mês e saber até quando está bloqueado (bloqueadoAte).

2. Elementos do Diagrama de Classes

2.1 Classe

Uma classe é um molde que descreve um conjunto de objetos com características e comportamentos em comum. “Cliente” descreve todos os clientes do TechBurger, não um cliente específico. O nome da classe é um substantivo, no singular, do vocabulário do negócio. Na notação UML, a classe é um retângulo dividido em 3 compartimentos:

Notação UML: um retângulo com 3 compartimentos



1 Nome da classe

substantivo, singular, começa com maiúscula

2 Atributos

os dados que cada objeto guarda

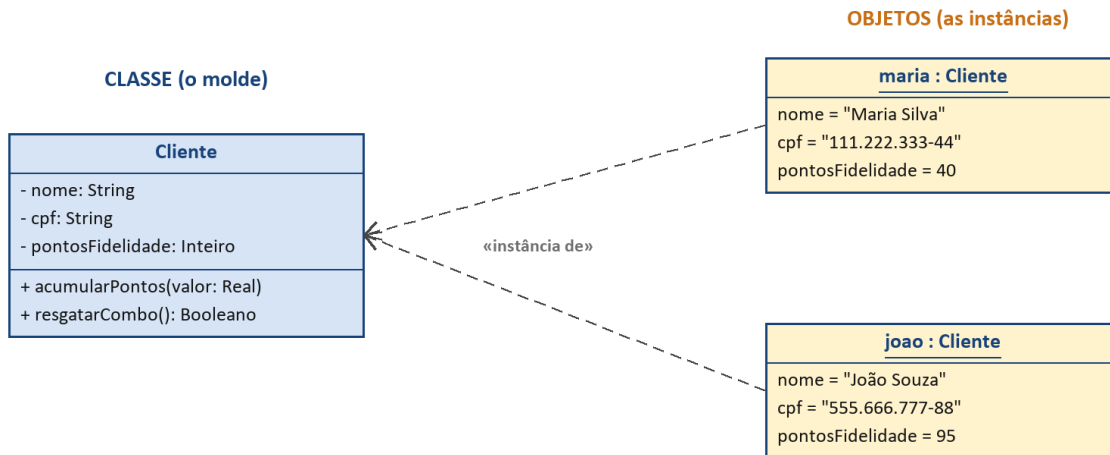
3 Métodos

o que a classe sabe fazer (verbo + parênteses)

Notação UML de uma classe: nome, atributos e métodos.

2.2 Objeto

Um objeto é um elemento concreto criado a partir de uma classe (uma instância). “maria” e “joao” são objetos da classe Cliente: têm a mesma estrutura (nome, cpf, pontos), mas valores diferentes. No Diagrama de Classes desenhamos a CLASSE, não os objetos.



Mesma estrutura, valores diferentes.

Classe (o molde) e objetos (as instâncias).

- Analogia: a forma de bolo é a classe; cada bolo que sai da forma é um objeto. Mudar a forma muda todos os bolos futuros.
- TechBurger: a classe Pedido tem os objetos “pedido nº 101 (R\$ 42,00, pago)” e “pedido nº 102 (R\$ 18,50, pendente)”.
- Biblioteca “Ler Mais”: a classe Livro tem os objetos “Dom Casmurro” (Machado de Assis) e “O Cortiço” (Aluísio Azevedo), com a mesma estrutura: título, autor e isbn.

2.3 Atributos

Atributos são as características (dados) que os objetos da classe guardam. Ficam no 2º compartimento e seguem o formato visibilidade nome: tipo — por exemplo, - preco: Real. Nomes em substantivo, sem espaços nem acentos, começando com minúscula (dataHora, qtdCancelamentosMes).

Tipo	Para que serve	Exemplo
String	Texto	nome, cpf, endereco
Inteiro	Números sem vírgula	quantidade, pontosFidelidade
Real	Números com vírgula	preco, valorTotal
Booleano	Verdadeiro ou falso	ativo
Data / DataHora	Datas e horários	dataEmprestimo, dataHora

Nesta disciplina o tipo é opcional no diagrama, mas ajuda quem for programar e criar o banco de dados depois.

Atributo derivado (/)

Um atributo derivado é um valor que pode ser calculado a partir de outros dados. Ele leva uma barra (/) antes do nome — por exemplo, - /valorTotal: Real em Pedido (a soma dos ItemPedido, calculada por calcularValorTotal()), - /qtdCancelamentosMes em Cliente (contagem dos pedidos cancelados no mês) e - /livrosEmprestados em Associado (empréstimos ainda não devolvidos). Pergunta para discussão: se guardarmos o valor e também o calcularmos, o que acontece quando os dois divergem?

Quiz rápido: classifiquem cada atributo abaixo como derivado ou não (respostas no Gabarito).

1. - valorTotal: Real (Pedido)
2. - /valorTotal: Real (Pedido)
3. - dataNascimento: Data (Cliente)
4. - /qtdCancelamentosMes: Inteiro (Cliente)
5. - /livrosEmprestados: Inteiro (Associado)
6. - matricula: String (Associado)

2.4 Métodos

Métodos são os comportamentos (ações) que os objetos da classe sabem executar. Ficam no 3º compartimento e seguem o formato visibilidade nome(parâmetros): retorno — por exemplo, + acumularPontos(valor: Real). O nome é um verbo: calcularSubtotal(), confirmar(), validarRaio(). Os verbos dos casos de uso viram candidatos a método.

Como ler um atributo e um método

Atributo

- isbn : String
visibilidade nome tipo

Método

+ trocarEstante (estante: Estante) : Booleano
visibilidade nome do método parâmetros retorno

Tipo e retorno são opcionais na disciplina, mas ajudam quem for programar depois.

Como ler um atributo e um método.

2.5 Atributo ou método? O teste

- É um dado que o objeto guarda (responde “o que ele TEM?”) → atributo. Ex.: pontosFidelidade, valorTotal, status.
- É uma ação que o objeto executa (responde “o que ele FAZ?”) → método. Ex.: acumularPontos(), calcularValorTotal().

Mini-exercício: classifiquem cada item como atributo ou método (respostas no Gabarito).

1. status
2. calcularSubtotal()
3. distanciaKm
4. precoUnitario
5. resgatarCombo()
6. dataHora
7. validarRaio()
8. confirmar()

2.6 Nomeando classes, atributos e métodos

- Classe: substantivo no singular, iniciais maiúsculas (ItemPedido, e não “itens_pedidos”).
- Atributo: substantivo, começa com minúscula (dataHora, qtdCancelamentosMes).
- Método: verbo, começa com minúscula, com parênteses (calcularSubtotal()).
- Sem acentos, espaços ou caracteres especiais: facilita virar código depois.
- Use o mesmo vocabulário dos casos de uso e das regras de negócio.

Da UML ao código: por que a visibilidade importa

A visibilidade do diagrama não é só um detalhe estético: ela vira uma palavra-chave da linguagem de programação. O símbolo + (público) vira public: qualquer classe pode acessar. O símbolo - (privado) vira private: só a própria classe acessa. O símbolo # (protegido) vira protected: a classe e suas subclasses acessam. A diferença importante é que, no código, o compilador passa a IMPEDIR o acesso indevido - o encapsulamento deixa de ser um combinado entre programadores e vira uma regra garantida pela linguagem. Por isso vale a pena decidir a visibilidade certa já no diagrama, antes de programar.

Exemplo - a classe Pedido em pseudocódigo:

```
classe Pedido {  
  
  - valorTotal: Real  
  
  - status: String  
  
  + confirmar(): void  
  
  + calcularValorTotal(): Real  
  
}
```

O diagrama e o código guardam a mesma informação - só muda a notação.

3. Visibilidade

A visibilidade define quem pode acessar um atributo ou método. Aparece como um símbolo antes do nome e protege os dados da classe contra alterações indevidas (encapsulamento).

Símbolo	Nome	Quem acessa	Exemplo
---------	------	-------------	---------

+	Público	Qualquer classe	+ confirmar() em Pedido
-	Privado	Só a própria classe	- pontosFidelidade em Cliente
#	Protegido	A própria classe e suas subclasses	# cargo em Funcionario

Uma classe com os três tipos

Funcionario
- cpf: String
cargo: String
+ nome: String
+ getNome(): String
calcularBonus(): Real
- validarCpf(): Booleano



público

qualquer classe acessa



privado

só a própria classe acessa



protegido

a classe e suas subclasses acessam

Boa prática: atributos privados (-), métodos de uso externo públicos (+).

Uma classe com os três tipos de visibilidade.

Regra de bolso: atributos privados (-), métodos de uso externo públicos (+).

Por que atributos privados? (encapsulamento)

Se pontosFidelidade fosse público, qualquer parte do sistema poderia gravar 500 pontos, violando a Regra 2 (1 ponto a cada R\$ 10,00 gastos). Com o atributo privado e o método acumularPontos(), a regra vive em UM lugar só: se ela mudar (por exemplo, 1 ponto a cada R\$ 8,00), muda em um único método. Na Biblioteca “Ler Mais”, o atributo - dataPrevista do Empréstimo só muda pelo método + renovar(), que aplica a regra do prazo.

Erros comuns com visibilidade

- Deixar tudo público (+) “porque é mais fácil”.
- Esquecer o símbolo: o diagrama fica ambíguo.
- Usar # em classe que não participa de herança.
- Deixar privado (-) um método que outras classes precisam chamar.

Quiz — qual símbolo?

1. Atributo cpf do Cliente, que ninguém de fora deve alterar.
2. Método confirmar() do Pedido, chamado pela tela do aplicativo.
3. Atributo cargo, usado por Atendente e Gerente (subclasses) mas por mais ninguém.
4. Método validarCpf(), usado só dentro da própria classe Funcionario.
5. Atributo dataPrevista do Empréstimo, que só deve mudar quando o empréstimo é renovado.

4. Identificando classes candidatas a partir de um texto

4.1 A técnica dos substantivos

1. Releia a descrição do sistema (regras de negócio, entrevistas, casos de uso).

2. Sublinhe os substantivos relevantes.
3. Agrupe os sinônimos (cliente = comprador).
4. Descarte o que não vira classe (valores soltos, o próprio sistema, detalhes irrelevantes).
5. O que sobra são as classes candidatas.
6. Depois, olhe os verbos: eles sugerem os métodos.

4.2 Nem todo substantivo vira classe

- Sinônimos: “cliente” e “comprador” são a mesma classe.
- Valores soltos: “R\$ 15,00” e “5 km” são valores de atributos ou regras, não classes.
- O próprio sistema (“a loja”, “o app”) costuma ser o contexto, não uma classe.
- Substantivo sem dados nem comportamentos próprios: provavelmente é atributo de outra classe. Ex.: “ponto” de fidelidade é atributo de Cliente (pontosFidelidade).

Teste: tem dados e comportamentos próprios? → classe. É só um valor que descreve outra coisa? → atributo. Exemplos: “Pagamento” tem forma, valor, status e confirma → classe; “forma de pagamento” (PIX, cartão) é só um valor → atributo de Pagamento. “Endereço” pode ser atributo (texto simples) ou classe (se for preciso validar e guardar vários por cliente).

4.3 Aplicando no texto do TechBurger

No texto da Aula 1, os substantivos que mais aparecem são cliente, pedido, produto (item do cardápio), pagamento, entrega, funcionário (atendente, gerente):

Trecho do texto	Substantivos candidatos
Pedidos feitos pelo aplicativo só confirmam com valor total ≥ R\$ 15,00	pedido, aplicativo, valor total
Fidelidade: 1 ponto a cada R\$ 10,00; 100 pontos = combo grátis	cliente, programa de fidelidade, ponto, combo
Pagamento confirmado → pedido vai para a cozinha	pagamento, cozinha
PIX no aplicativo tem 5% de desconto	pagamento (PIX), desconto
Entrega só até 5 km de distância	entrega, endereço, distância
Item em falta: sugerir substituto	item do cardápio (produto), estoque, substituto
Mais de 10 itens exige confirmação do gerente	pedido, item, gerente
3 cancelamentos no mês bloqueiam o app por 15 dias	cliente, pedido, cancelamento

Viram classe	Viram atributo ou são descartados
Cliente, Pedido, ItemPedido, Produto, Pagamento, Entrega, Funcionario (com Atendente e Gerente)	valor total → atributo de Pedido; ponto → atributo de Cliente; estoque → atributo de Produto; distância → atributo de Entrega; PIX e cartão → valores de Pagamento.forma; aplicativo, loja, cozinha → canal, contexto ou ator

4.4 E os verbos? Descobrimos métodos

- Confirmar pedido → Pedido.confirmar()
- Acumular pontos → Cliente.acumularPontos()
- Validar raio de entrega → Entrega.validarRaio()
- Verificar item em falta → Produto.verificarDisponibilidade()

- Autorizar pedido grande → `Gerente.autorizarPedidoGrande()`

4.5 Aplicando na Biblioteca “Ler Mais”

Texto (Aula 1): “Cada associado pode manter até 3 livros emprestados ao mesmo tempo. O prazo padrão de empréstimo é de 14 dias, mas associados classificados como ‘professor’ têm prazo estendido para 30 dias. Quando um livro é devolvido com atraso, é cobrada uma multa de R\$ 1,00 por dia de atraso, exceto para livros reservados para leitura escolar.”

- Classes candidatas: Associado, Livro, Emprestimo e Professor (um tipo de Associado).
- Regras que viram método: `Emprestimo.calcularMulta()` (R\$ 1,00 por dia de atraso), `Associado.podePegarLivro()` (limite de 3 livros) e `Associado.prazoEmprestimo()` (14 dias; o Professor redefine para 30).
- Atributos que surgem do texto: as datas do empréstimo e uma marcação de leitura escolar no Livro (sem multa).

4.6 Mini-exercício: academia FitLife

“A academia FitLife tem alunos matriculados em planos (mensal ou trimestral). Cada aluno pode reservar vagas em aulas coletivas, ministradas por instrutores. Cada aula tem horário, sala e limite de vagas. A mensalidade é paga por boleto ou cartão.”

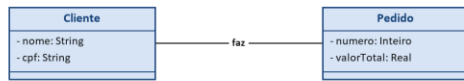
Sublinhem os substantivos e separem em três listas: classes candidatas, atributos e descartados. Respostas no Gabarito.

5. Relacionamentos básicos entre classes

Classes soltas não contam a história do negócio. Os relacionamentos nascem dos verbos do texto (“o cliente FAZ um pedido”). Hoje vemos apenas o conceito de cada relacionamento; a cardinalidade (1, 0..1, 1..*, *) é o foco da Aula 6.

Relacionamento	Significado	Notação	Exemplo
Associação	As classes se conhecem e colaboram	Linha simples (com verbo)	TechBurger: Cliente faz Pedido
Agregação	Todo-parte “fraca”: a parte existe sem o todo	Losango vazado no lado do todo	Biblioteca: Estante reúne Livros
Composição	Todo-parte “forte”: a parte não existe sem o todo	Losango preenchido no lado do todo	TechBurger: Pedido é composto por ItemPedido
Herança	“É um tipo de”: a subclasse herda da superclasse	Seta com triângulo vazado, apontando para a superclasse	Atendente e Gerente são tipos de Funcionario

Associação — linha simples



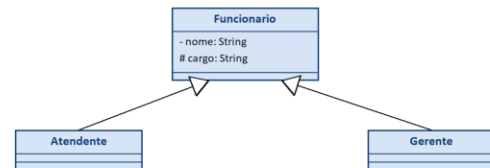
Composição — losango preenchido (parte não existe sem o todo)



Agregação — losango vazado (parte existe sem o todo)



Herança — seta com triângulo vazado ("é um tipo de")



Os quatro relacionamentos básicos.

Agregação ou composição? O teste

Pergunte: se o todo deixar de existir, a parte também deixa de fazer sentido? Sim → composição (um ItemPedido sem Pedido não faz sentido). Não → agregação (um Livro continua existindo sem a Estante). Na dúvida, comece com uma associação simples.

Mais alguns exemplos para fixar (respostas no Gabarito):

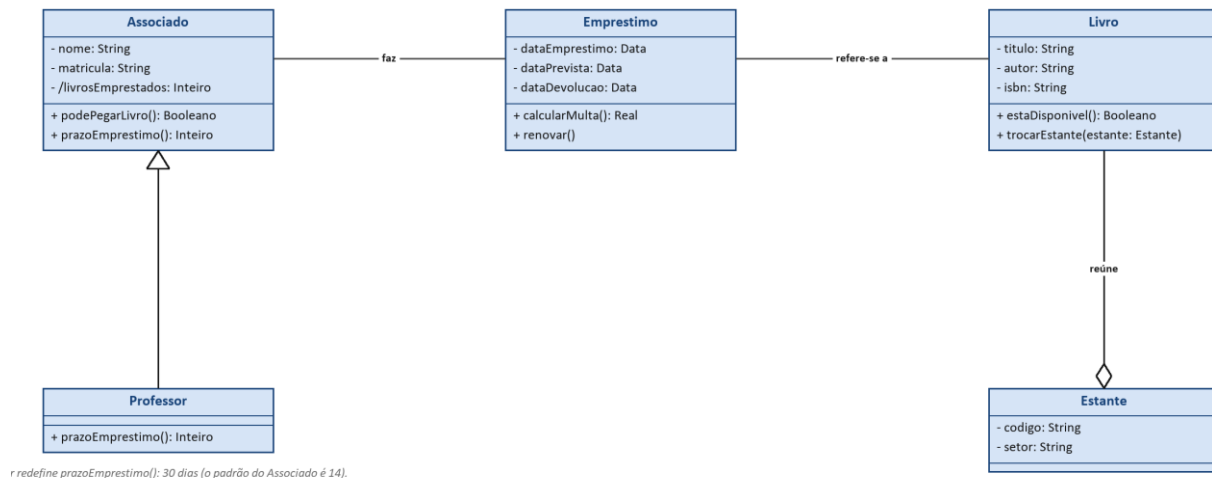
1. TechBurger: ItemPedido existe sem Pedido?
2. TechBurger: Entrega existe sem Pedido?
3. Biblioteca: Livro existe sem Estante?
4. Biblioteca: Emprestimo existe sem Associado?
5. TechBurger: Produto existe sem Pedido?

Herança: quando usar

- Quando dá para dizer "X é um tipo de Y": Atendente é um Funcionario.
- A subclasse herda tudo da superclasse e só declara o que é próprio dela (Atendente tem registrarPedidoBalcao(); Gerente tem autorizarPedidoGrande()).
- Cuidado: não use herança só para "reaproveitar código". "Pedido herda de Cliente" está errado: pedido não é um tipo de cliente.

Relacionamentos no TechBurger e na Biblioteca

- TechBurger: Cliente faz Pedido; Pedido gera Pagamento; Pedido possui Entrega; ItemPedido refere-se a Produto; Atendente registra Pedido; Gerente autoriza Pedido (associações); Pedido é composto por ItemPedido (composição); Atendente e Gerente são tipos de Funcionario (herança).
- Biblioteca: Associado faz Emprestimo; Emprestimo refere-se a Livro (associações); Estante reúne Livros (agregação); Professor é um tipo de Associado (herança).



Biblioteca “Ler Mais”: Diagrama de Classes de exemplo.

Quiz — que relacionamento é esse?

1. Um Cliente faz Pedidos.
2. Cancelando o Pedido, os seus ItemPedido também deixam de existir.
3. Professor é um tipo de Associado.
4. Uma Estante reúne Livros, mas um Livro pode ir para outra estante.
5. Um Pedido gera um Pagamento.

Retomem a academia FitLife (seção 4.6) e pensem agora nos relacionamentos entre as classes que vocês já identificaram (respostas no Gabarito):

1. Aluno e Plano: que relacionamento?
2. Aula e Instrutor: que relacionamento?
3. Aluno e Aula, feito através da classe Reserva: que relacionamento?
4. Se a Aula fosse cancelada, a Reserva ainda faria sentido?

6. O cenário TechBurger — classes para modelar

Estas são as classes do TechBurger identificadas a partir do texto da Aula 1 e dos casos de uso da Aula 4. Os atributos e métodos NÃO estão nesta apostila de propósito: a atividade da seção 9 consiste em derivá-los das regras de negócio e dos casos de uso, e em justificar cada escolha. A coluna “onde procurar” indica as pistas.

Classe	Papel no negócio	Onde procurar atributos e métodos
Cliente	Quem faz os pedidos e acumula pontos	Regras 2 e 8; casos de uso Acumular Pontos, Resgatar Combo Grátis e Recusar Pedido de Cliente Bloqueado
Pedido	O pedido feito no app ou no balcão	Regras 1, 3, 4 e 7; casos de uso Fazer Pedido pelo App e Fazer Pedido no Balcão
ItemPedido	Cada linha de um pedido	O que compõe um pedido: um produto e uma quantidade
Produto	Item do cardápio	Regra 6; caso de uso Sugerir Item Substituto
Pagamento	O pagamento de um pedido	Regras 3 e 4; casos de uso Confirmar Pagamento e Aplicar

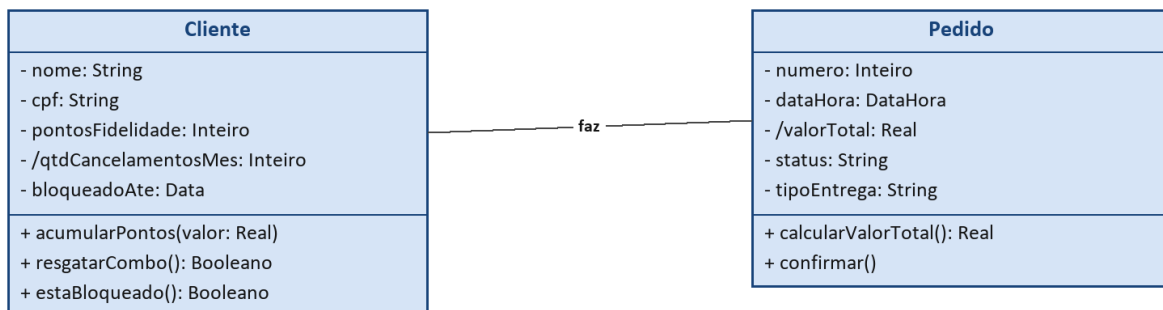
		Desconto PIX
Entrega	A entrega de um pedido	Regra 5; caso de uso Validar Endereço de Entrega
Funcionario, Atendente, Gerente	Quem trabalha na loja	Generalização da Aula 4; casos de uso Fazer Pedido no Balcão e Autorizar Pedido Grande

Lembretes: atributos calculáveis a partir de outros dados levam / (derivado); Atendente e Gerente são subclasses de Funcionario; e os casos de uso da Aula 4 também sugerem associações (quem registra o pedido? quem o autoriza?).

7. Exemplo guiado: Cliente e Pedido

Vamos desenhar juntos, ao vivo, as classes Cliente e Pedido no draw.io:

1. No painel de formas, ative UML (Mais formas → UML) e arraste a forma “Class”.
2. Dê o nome da classe no compartimento de cima.
3. Escreva os atributos no compartimento do meio, um por linha, com visibilidade e tipo.
4. Escreva os métodos no compartimento de baixo.
5. Repita para a segunda classe.
6. Ligue as duas com uma linha (associação) e escreva o verbo sobre ela.



Exemplo guiado: Cliente faz Pedido.

Checklist: as 3 partes de cada classe estão preenchidas? Todo atributo e método tem visibilidade? Os atributos têm tipo? Os nomes das classes estão no singular? A associação tem um verbo que faz sentido? Cada atributo pode ser justificado por uma regra ou caso de uso?

O mesmo exemplo na Biblioteca “Ler Mais”: a classe Associado (nome, matricula, /livrosEmprestados; podePegarLivro(), prazoEmprestimo()) se associa à classe Emprestimo (dataEmprestimo, dataPrevista, dataDevolucao; calcularMulta()) pelo verbo “faz”.

8. Erros comuns ao modelar Classes

Erro comum	Correção
Confundir agregação com composição	Teste: se o todo some, a parte some junto? Composição só para Pedido — ItemPedido; Estante — Livro é agregação.
Esquecer visibilidade e tipo	Escrever “- nome: String” e “+ calcularValorTotal(): Real”, e não apenas

	"nome" e "calcularValorTotal()".
Nomes no plural, com espaço ou como verbo	Classe é substantivo no singular (Pedido, ItemPedido); verbos são métodos, não classes ("FazerPedido" está errado).
Transformar tudo em classe	"Nome", "Valor" e "CPF" são atributos de Cliente e Pedido, não classes.
Classe "faz-tudo"	Uma classe "Sistema" ou "Gerenciador" com 40 métodos: distribua as responsabilidades entre classes pequenas e coesas.
Herança invertida ou só para reaproveitar código	A seta sai da subclasse (Atendente) e aponta para a superclasse (Funcionario); use herança só quando "X é um tipo de Y" fizer sentido.
Relacionamento escondido como atributo	Não coloque "clienteld" em Pedido: desenhe uma linha de associação entre Cliente e Pedido.

9. Atividade prática 1 — TechBurger (em grupo)

- Em grupo, desenhem (no draw.io ou papel) o Diagrama de Classes do TechBurger com pelo menos 5 classes da seção 6, derivando atributos e métodos das regras e dos casos de uso, com tipos e visibilidade.
- Representem 1 herança (Funcionario ← Atendente e Gerente) e 1 composição (Pedido — ItemPedido), além de pelo menos 2 associações com verbo sobre a linha.
- Partam dos substantivos do texto e dos verbos dos casos de uso.
- Cardinalidade fica para a Aula 6: hoje não é preciso escrever números nas linhas.
- Marquem os atributos calculáveis como derivados (/) e estejam prontos para justificar cada atributo, método e relacionamento.
- Tempo: 35 minutos. Ao final, um representante do grupo apresenta o diagrama para a turma.

10. Atividade prática 2 — Academia FitLife (individual)

Agora sozinho(a): apliquem o mesmo processo à academia FitLife (texto da seção 4.6). A tabela abaixo não traz atributos e métodos prontos - vocês devem derivá-los do texto e da resposta da seção 4.6, do mesmo jeito que fizeram com o TechBurger.

Classe	Papel no negócio	Onde procurar atributos e métodos
Aluno	Pessoa matriculada na academia	Seção 4.6: quem se matricula, reserva vaga e paga a mensalidade
Plano	Modalidade e valor da mensalidade	Seção 4.6: "matriculados em planos (mensal ou trimestral)"
Aula	A aula coletiva, com horário e vagas	Seção 4.6: "cada aula tem horário, sala e limite de vagas"
Instrutor	Quem ministra a aula	Seção 4.6: "aulas coletivas, ministradas por instrutores"
Reserva	A vaga de um Aluno em uma Aula específica	Seção 4.6: "cada aluno pode reservar vagas em aulas"
Pagamento	A mensalidade paga por boleto ou cartão	Seção 4.6: "a mensalidade é paga por boleto ou cartão"

- Desenhem (no draw.io ou papel) o Diagrama de Classes completo: cada classe com nome, atributos (visibilidade + tipo) e métodos.
- Representem pelo menos 3 associações e 1 composição (Reserva — Aula).
- Esta atividade não tem herança: nem todo domínio precisa de subclasses.

- Cardinalidade fica para a Aula 6: hoje não é preciso escrever números nas linhas.
- Marquem os atributos calculáveis como derivados (/).
- Tempo: 20 minutos, individual. Ao final, comparem com um colega antes da correção.

Resposta-modelo — Academia FitLife (comparem depois de tentar sozinhos):

- Aluno: - nome, - cpf, - /pagamentosEmDia; + matricular(), + reservarVaga()
- Plano: - tipo, - valorMensalidade; + calcularValor()
- Aula: - horario, - sala, - limiteVagas, - /vagasDisponiveis; + temVagaDisponivel()
- Instrutor: - nome, - especialidade; + ministrar()
- Reserva: - dataReserva, - status; + cancelar()
- Pagamento: - valor, - formaPagamento, - dataPagamento; + confirmar()

Leitura da resposta: as associações são Aluno-Plano (está matriculado), Aluno-Reserva (faz), Instrutor-Aula (ministra) e Aluno-Pagamento (realiza); a composição é Reserva-Aula, porque sem a Aula a Reserva não faz sentido (losango preenchido do lado da Aula). Os atributos /pagamentosEmDia (Aluno) e /vagasDisponiveis (Aula) são derivados: calculados, não guardados diretamente.

11. Exercício: encontre o erro

Em duplas, analisem o diagrama abaixo. Ele tem 6 erros propositais. Para cada um: (1) marquem onde está, (2) expliquem POR QUE é um erro — qual conceito da aula foi violado — e (3) escrevam como ficaria a correção. Não consultem a Apostila nos primeiros 5 minutos. Respostas no Gabarito.

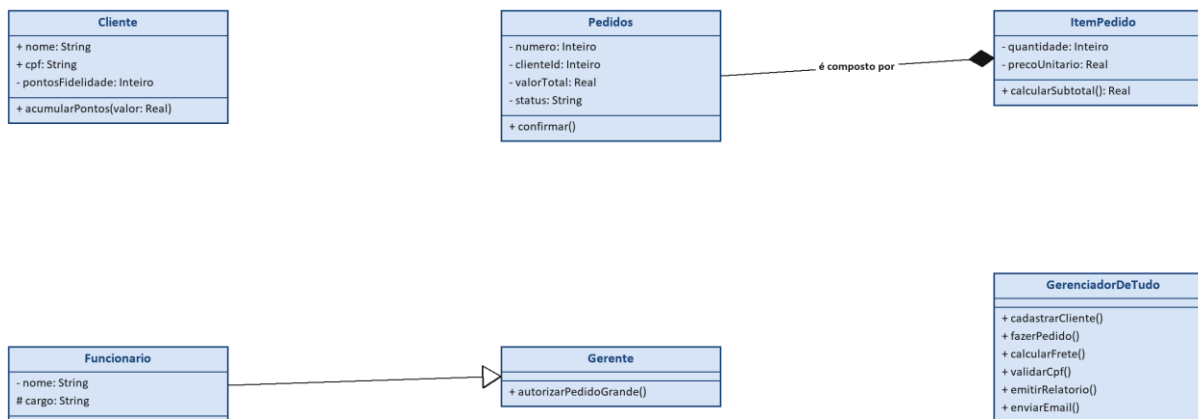


Diagrama de classes com 6 erros propositais.

Glossário

Termo	Significado
Classe	Molde que define atributos e métodos comuns a um conjunto de objetos.
Objeto	Instância concreta de uma classe, com valores próprios.
Atributo	Característica (dado) guardada pela classe.
Método	Comportamento (ação) que a classe executa.
Visibilidade	Quem pode acessar: + público, - privado, # protegido.
Classe candidata	Substantivo do cenário que pode virar classe.

Associação	Relação simples entre classes (linha reta).
Agregação	Todo-parte fraca: a parte existe sem o todo (losango vazado).
Composição	Todo-parte forte: a parte não existe sem o todo (losango preenchido).
Atributo derivado	Atributo calculável a partir de outros dados; leva / antes do nome (ex.: /valorTotal).
Herança	Relação “é um tipo de” entre subclasse e superclasse (triângulo vazado).

Perguntas de revisão

1. Quais são as 3 partes de uma classe na notação UML?
2. Qual a diferença entre classe e objeto? Dê um exemplo do TechBurger.
3. O que significa o símbolo “-” antes de um atributo? E o “#”?
4. Como a técnica dos substantivos ajuda a achar classes candidatas?
5. Diferencie agregação de composição com um exemplo de cada.
6. Por que “Pedido herda de Cliente” é um erro de modelagem?
7. Na Biblioteca “Ler Mais”, por que Professor é uma herança de Associado?
8. Por que os atributos costumam ser privados e os métodos públicos?
9. Por que a visibilidade do diagrama vira public/private/protected no código?

Quiz relâmpago para casa

Cinco perguntas para conferir o que ficou. Respostas no Gabarito.

1. Quais são as 3 partes de uma classe?
2. No atributo “- pontosFidelidade: Inteiro”, o que são “-”, “pontosFidelidade” e “Inteiro”?
3. Qual a diferença entre a classe Cliente e o objeto maria : Cliente?
4. Pedido e ItemPedido: agregação ou composição? Por quê?
5. Em Funcionario ← Gerente, qual é a subclasse? Que símbolo deixa só as subclasses acessarem?